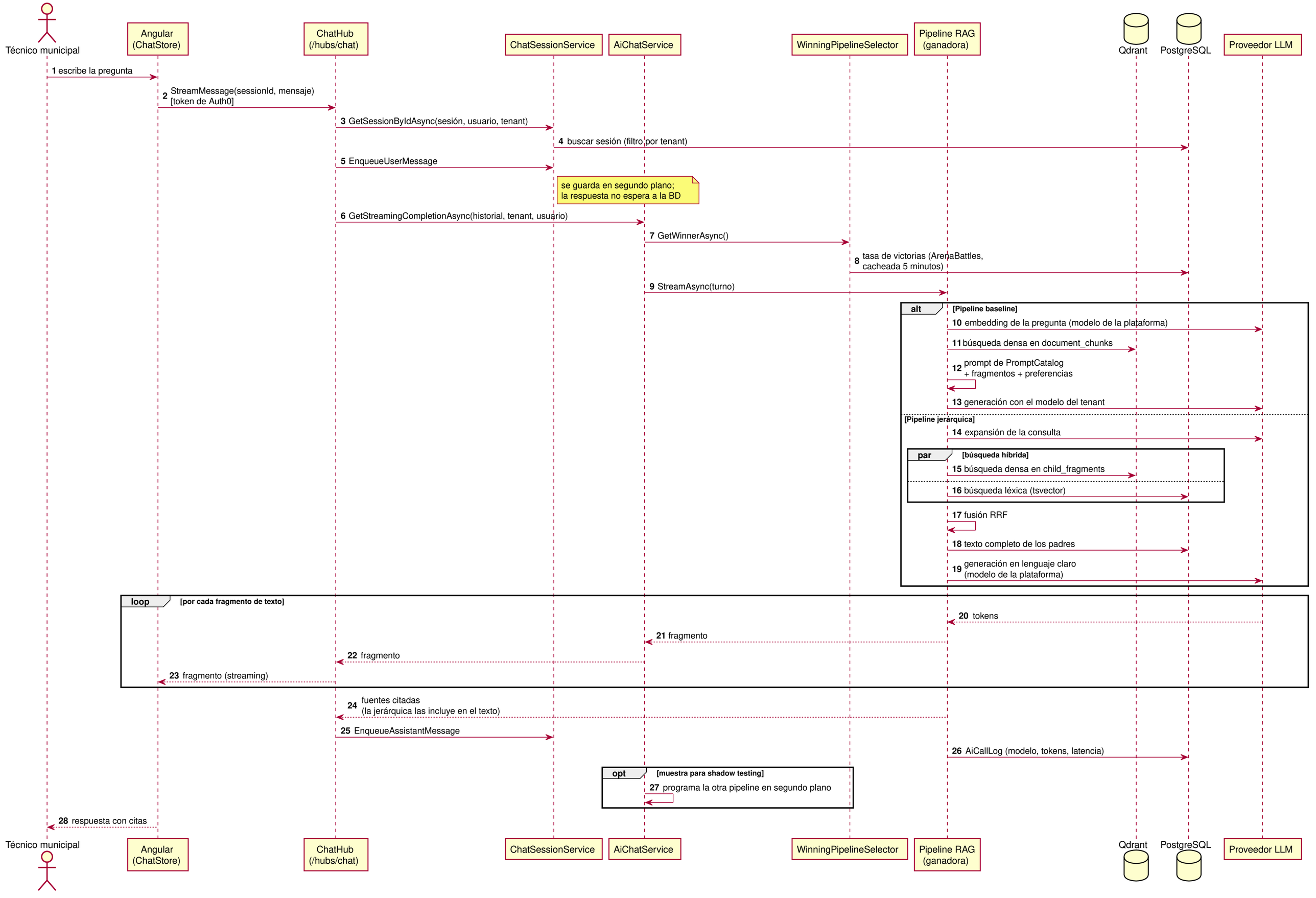
Técnico municipal
Angular (ChatStore)
ChatHub (/hubs/chat)
ChatSessionService
AiChatService
WinningPipelineSelector
Pipeline RAG (ganadora)
Qdrant
PostgreSQL
Proveedor LLM
1 escribe la pregunta
2 StreamMessage(sessionId, mensaje) [token de Auth0]
3 GetSessionByIdAsync(sesión, usuario, tenant)
4 buscar sesión (filtro por tenant)
5 EnqueueUserMessage
se guarda en segundo plano; la respuesta no espera a la BD
6 GetStreamingCompletionAsync(historial, tenant, usuario)
7 GetWinnerAsync()
8 tasa de victorias (ArenaBattles, cacheada 5 minutos)
9 StreamAsync(turno)
alt
[Pipeline baseline]
10 embedding de la pregunta (modelo de la plataforma)
11 búsqueda densa en document_chunks
12 prompt de PromptCatalog + fragmentos + preferencias
13 generación con el modelo del tenant
[Pipeline jerárquica]
14 expansión de la consulta
par
[búsqueda híbrida]
15 búsqueda densa en child_fragments
16 búsqueda léxica (tsvector)
17 fusión RRF
18 texto completo de los padres
19 generación en lenguaje claro (modelo de la plataforma)
loop
[por cada fragmento de texto]
20 tokens
21 fragmento
22 fragmento
23 fragmento (streaming)
24 fuentes citadas (la jerárquica las incluye en el texto)
25 EnqueueAssistantMessage
26 AiCallLog (modelo, tokens, latencia)
opt
[muestra para shadow testing]
27 programa la otra pipeline en segundo plano
28 respuesta con citas